

Lab 8: Arithmetic Circuits and Behavioral VHDL

In this lab, you will learn the following:

- How to interpret synthesized VHDL netlists
- How to write behavioral VHDL
- How to design and implement arithmetic circuits

Part 1: Synthesis of a Ripple-Carry Adder

In this part, you will design an 8-bit Ripple carry adder circuit and you will examine the results from synthesizing your VHDL using the netlist viewers in Quartus.

1. On the BB assignment page for this lab are VHDL files for a 4-bit ripple carry adder. Start by examining each of these files:
 - **Full_Adder.vhd** – This VHDL file contains a gate-level description of a full adder circuit. The VHDL file implements the circuit using only 2-input logic gates.
 - **Four_Bit_RCA.vhd** – This file instantiates 4 full adder components and connects them using port maps to create a ripple carry adder. Note that in port maps, using the `=>` operator is recommended to eliminate confusion that may occur when declaring mappings via the ordering of the signal list.
 - **Lab8.vhd** – This is an example top-level file that instantiates one copy of the 4-bit adder. Note that in this file, the DE10-Lite dev board parts are not referenced specifically (e.g. SW1). The mapping of VHDL signals to FPGA resources should occur at the pin planner step. Writing the top-level VHDL in this way will help you to avoid confusion between the names of FPGA resources and names of signals that are relevant to your design. You should follow a similar format each time you are asked to create a new top-level entity.
2. Sketch a schematic that represents the full adder logic network as implemented in **Full_Adder.vhd**. Include this schematic in your report.
3. Calculate the cost to implement an 8-bit ripple carry adder circuit using the full adder implementation from the previous step. Use the cost calculation from class, where $\text{Cost} = (\text{number of gates}) + (\text{number of inputs to each gate})$.
4. Create a new Quartus project for this lab, and add a new VHDL file named `lab8_part1.vhd`. Make this file the top-level entity for your project.
5. Add `Full_Adder.vhd` to the project.

6. Design an 8-bit ripple carry adder using multiple instances of the full adder circuit. You may do this by either instantiating 8 copies of the full adder circuit OR 2 copies of the 4-bit ripple carry adder example circuit. Write VHDL to implement the 8-bit adder in your file lab8_part1.vhd.
7. Select “Start Analysis and Synthesis” to compile your circuit
8. Go to “Tools→Netlist Viewers→RTL Viewer”. Examine the schematic and think about how it corresponds to the VHDL code that you wrote (Note: you can expand component blocks by double clicking on them in the RTL viewer).
9. Take a screenshot of the synthesized logic network for the 8-bit adder circuit (from the RTL viewer). Include this screenshot in your report.
10. Calculate the cost of the logic circuit as shown in the RTL viewer and compare the result to the cost you calculated by hand previously. Briefly explain your results, and comment on the significance of these results to creating large scale digital circuit designs.
11. Go to “Tools→Netlist Viewers→Technology Map Viewer”. Include a screenshot in your report. You will use this to compare to another implementation later in this lab.

Part 2: Behavioral VHDL MUX

In this part, you will practice writing **behavioral VHDL code** to implement a multiplexer (MUX).

1. Read section 5.2 and 5.3 of “Free Range VHDL” (linked on BB under Quartus, ModelSim, and DE10-Lite Tutorials and Resources). These sections provide a brief background on behavioral VHDL and the **PROCESS** statement. “Behavioral” VHDL allows us to describe a circuit at an abstract level, without specifying a gate-level implementation.
2. Section 5.4.2 explains how to use the **IF** statement and section 5.4.3 explains how to use the **CASE** statement. Quickly glance through these sections. Both sections contain examples of an 8:1 mux being implemented using these statements (Listing 5.8 and 5.9). Review these examples and note how the VHDL is written.
3. Using one of these examples as a guide, design a **4-bit by 4:1 MUX** using either **IF or CASE statements**. Write your VHDL for the MUX in a new vhd file, and make it your top-level entity.
4. Run Analysis and Synthesis and use the RTL viewer to examine the resulting schematic. Include a screenshot in your report.
5. Use the Technology Map Viewer to inspect the implementation. Include a screenshot in your report.
6. Compare this synthesis result to your previous implementation of this MUX in Lab 7. Other than the logic functions themselves, are there any significant differences in how the compiler decided to implement the two designs?
7. **Read section 5.5 of the Free Range VHDL text** (it is brief and will serve you well if you have future plans in digital hardware / FPGA design).

Part 3: VHDL Arithmetic

In this part, you will practice utilizing VHDL arithmetic libraries to implement an adder circuit.

1. VHDL supports several high-level operators, such as + for addition, included in VHDL arithmetic libraries. Reference the code examples from lecture showing how to add numbers in VHDL. Note that addition is performed on UNSIGNED types, and you can **cast** from STD_LOGIC_VECTOR to UNSIGNED and vice versa. Signals must be the same number of bits to be added together with the + operator, and you can concatenate bits using the & operator using the syntax: '0'&A
2. Use the IEEE.NUMERIC_STD library and the + operator to design an 8-bit binary adder. Write VHDL to implement this adder in a new vhd file and make it your top-level entity.
3. Run analysis and synthesis and use the RTL viewer to examine the resulting schematic. Include a screenshot in your report.
4. Use the Technology Map Viewer to inspect the implementation. Include a screenshot in your report.
5. Compare this synthesis result to the previous ripple carry adder synthesis results. Other than the logic functions themselves, are there any significant differences in how the compiler synthesized this adder circuit versus the previous adder? Right click on the logic blocks and select "Properties" to view more implementation detail. What does your result suggest about the internal structure of the FPGA (Hint: See Figure 1, Section 1.1.3.2.2 and figure 7 of the MAX10 FPGA Handbook linked on BB with the Quartus tutorials).

Part 4: Timing Simulation

In this section, you will model timing and effects of propagation delay using VHDL.

1. Assume that AND gates have a delay of 2ns, OR gates have a delay of 2ns, XOR gates have a delay of 4ns and NOT gates have a delay of 1ns. Use the VHDL **after** statement to incorporate these delays into the design of the full adder circuit from Part 1 (see lecture notes for an example).
2. Re-simulate your ripple carry adder, and measure the propagation delay for one stage of your ripple carry adder (i.e., measure the time from when the inputs change until when the carry-out for that stage reached steady state). Show your simulation result and report the delay.
3. Examine your ripple carry adder RTL schematic – what would you expect to be the propagation delay of one stage? How does this compare to your simulated propagation delay, and what does this tell you about the RTL viewer?
4. Calculate the expected critical path delay of your ripple-carry adder. Contemplate all the possible input/outputs to your adder and generate a test case (i.e., a set of values for the input signals) that relies on the critical path. Simulate this test case, show your simulation result and also report the delay that you measured.
5. Show your test case and VHDL implementation to an instructor or TA for the lab checkoff.